Software Requirements Specification for Software Engineering: subtitle describing software

Team #16, DevSquared
Omar Ghoneim
Meena Al Omran
Marko Popovic
Maya Azar
Amirah Abdelrahman

October 11, 2026

Contents

1 Introduction **1**
- 1.1 Purpose . . . 2
- 1.2 Scope . . . 2
- 1.3 Stakeholders . . . 2
- 1.4 Definitions and Acronyms . . . 3
- 1.5 Characteristics of Intended Reader . . . 3

2 System Overview **3**
- 2.1 Problem Statement . . . 3
- 2.2 Product Vision . . . 4
- 2.3 System Context . . . 4
- 2.4 User Characteristics . . . 5
- 2.5 System Constraints . . . 5

3 Stakeholder Requirements **5**

4 User Personas **6**
- 4.1 Persona 1 . . . 6
- 4.2 Persona 2 . . . 6

5 Use Cases **6**
- 5.1 UC1: Example Use Case . . . 7

6 Functional Requirements **7**

7 Data Requirements **7**
- 7.1 Major Data Entities . . . 8
- 7.2 Data Dictionary . . . 8
- 7.3 Privacy and Data Retention . . . 8

8 Non-Functional Requirements **8**

9 AI and Machine Learning Requirements **9**
- 9.1 Model Inputs . . . 9
- 9.2 Model Outputs . . . 9
- 9.3 Evaluation Metrics . . . 10
- 9.4 Known Limitations . . . 10

10 Requirements Traceability Matrices **10**

11 Rationale **10**

12 Likely Changes **11**

13 Unlikely Changes **11**

14 Appendices **12**
14.1 Appendix A: Additional Materials 12
14.2 Appendix B: Development Plan 12
14.3 Appendix C: Values of Auxiliary Constants 12
14.4 Appendix D: Generative AI Disclosure 12
14.5 Appendix E: Reflection 12

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[All capstone SRS documents should have a reflection appendix. —TPLT]

[This SRS template is based on the IEEE template, with input from co-pilot and S. Smith. It will get you started. You should not modify the section headings, without first discussing the change with the course instructor. Modification means you are not following the template, which loses some of the advantage of a template, especially standardization. —TPLT]

[Feel free to change the appearance of the report by modifying the LaTeX commands. —TPLT]

[This template document assumes that a single program is being documented. If you are documenting a family of models, you should start with a commonality analysis. —TPLT]

[The SRS is not generally written, or read, sequentially. The SRS is a reference document. It is generally read in an ad hoc order, as the need arises. —TPLT]

[Guiding principles for the SRS document:

- Do not repeat the same information at the same abstraction level. If information is repeated, the repetition should be at a different abstraction level.

—TPLT]

[The template description comments should be disabled before submitting this document for grading. —TPLT]

[You can borrow any wording from the text given in the template. It is part of the template, and not considered an instance of academic integrity. Of course, you need to cite the source of the template. —TPLT]

[When the documentation is done, it should be possible to trace back to the source of every piece of information. Some information will come from external sources, like terminology. —TPLT]

[An SRS document should have the following qualities: unambiguous, consistent, complete, validatable, abstract and traceable. —TPLT]

[The overall goal of the SRS is that someone can learn, understand and verify the captured domain knowledge. They should not have to trust the authors of the SRS on any statements. They should be able to independently verify/derive every statement made. —TPLT]

1 Introduction

TESTING SRS IEEE

[Explain what system is being specified. State purpose, users, and project goals. —TPLT]

[Introduce the name of your software as Software Engineering. Using Software Engineering as a command lets you change the name if necessary and have the change propagate to all documents. —TPLT]

[Include a reference to the template this work is based on. **?** justifies the SRS structure and **?** provides the broader life-cycle context that underlines modern requirements engineering practice. —TPLT]

1.1 Purpose

[State the purpose of the system. —TPLT]

[The purpose should be in the context of the project itself, not in the context of this course. Although the "purpose" of the document is to get a grade, you should not mention this. Instead, "fake it" as if this is a real project. —TPLT]

Software Engineering is intended to solve ... [What problem does your program solve? The description here should be in the problem space, not the solution space. —TPLT]

1.2 Scope

[Briefly describe:

- Problem being solved
- Intended users
- Major functionality
- Expected benefits
- Clarifying the scope often means saying what the software doesn't do

—TPLT]

[The scope section is relevant for understanding the boundaries of the system. It is also relevant for later determining typical values of inputs. The scope should make it clear what inputs are reasonable to expect. This is a distinction between scope and context (context is a later section). Scope affects the inputs while context affects how the software will be used. —TPLT]

TODO

1.3 Stakeholders

- Customer
- End Users
- Project Sponsor
- Development Team
- Administrators

1.4 Definitions and Acronyms

[This section provides the meaning of the different words and phrases used in the domain of the problem. —TPLT]

Term	Definition
TODO	TODO

Table 1: Definitions and Acronyms

1.5 Characteristics of Intended Reader

[This section summarizes the skills and knowledge of the readers of the SRS. It does NOT have the same purpose as the "User Characteristics" section (Section 2.4). The intended readers are the people that will read, review and maintain the SRS. They are the people that will conceivably design the software that is intended to meet the requirements. The user, on the other hand, is the person that uses the software that is built. They may never read this SRS document. Of course, the same person could be a "user" and an "intended reader." —TPLT]

[The intended reader characteristics should be written as unambiguously and as specifically as possible. Rather than say, the user should have an understanding of physics, say what kind of physics and at what level. For instance, is high school physics adequate, or should the reader have had a graduate course on advanced quantum mechanics? —TPLT]

2 System Overview

This section provides general information about the system. It identifies the problem, the vision, and the interfaces between the system and its environment. This section also describes user characteristics and lists the system constraints.

[The purpose of this section is to provide general information about the system so that the specific requirements in the later sections will be easier to understand. The general system description section is designed to be changeable independent of changes to the functional requirements documented in the specific system description. The general system description provides a context for a family of related models. The general description can stay the same, while specific details are changed between family members. —TPLT]

2.1 Problem Statement

[Current situation, problem, impact, desired improvement. You will likely borrow some text from your problem description document. —TPLT]

TODO

2.2 Product Vision

[A clear and concise statement of the product's purpose and goals. —TPLT]

[The goal statements are functional and nonfunctional objectives that system under consideration should achieve. Goals provide criteria for sufficient completeness of a requirements specification and for requirements pertinence. Goals will be refined by requirements. The goals are written abstractly, with a minimal amount of technical language. They should be understandable by non-domain experts. —TPLT]

[You will likely borrow from your goal statements in your problem description document. —TPLT]

TODO

2.3 System Context

[Include users, external systems, APIs, databases, and third-party services. —TPLT]

[Your system context will include a figure that shows the abstract view of the software. In some cases the diagram will show other external entities, besides the user. For instance, when the software product is a library, the user will be another software program, not an actual end user. If there are system constraints that the software must work with external libraries, these libraries can also be shown on the System Context diagram. They should only be named with a specific library name if this is required by the system constraint. —TPLT]



Figure 1: System Context

[For each of the entities in the system context diagram its responsibilities should be listed. Whenever possible the system should check for data quality, but for some cases the user will need to assume that responsibility. The list of responsibilities should be about the inputs and outputs only, and they should be abstract. Details should not be presented here. However, the information should not be so abstract as to just say "inputs" and "outputs". A summarizing phrase can be used to characterize the inputs. For instance, saying "material properties" provides some information, but it stays away from the detail of listing every required properties. —TPLT]

- User Responsibilities:
  - –
- Software Engineering Responsibilities:

- Detect data type mismatch, such as a string of characters instead of a floating point number
-

[Identify in what context the software will typically be used. Is it for exploration? education? engineering work? scientific work?. Identify whether it will be used for mission-critical or safety-critical applications. —TPLT]

[This additional context information is needed to determine how much effort should be devoted to the rationale section. If the application is safety-critical, the bar is higher. This is currently less structured, but analogous to, the idea to the Automotive Safety Integrity Levels (ASILs) that McSCert uses in their automotive hazard analyses. —TPLT]

2.4 User Characteristics

[This section summarizes the knowledge/skills expected of the user. Measuring usability, which is often a required non-function requirement, requires knowledge of a typical user. As mentioned above, the user is a different role from the "intended reader," as given in Section 1.5. As in Section 1.5, the user characteristics should be specific an unambiguous. For instance, "The end user of Software Engineering should have an understanding of undergraduate Level 1 Calculus and Physics." —TPLT]

2.5 System Constraints

[System constraints differ from other type of requirements because they limit the developers' options in the system design and they identify how the eventual system must fit into the world. This is the only place in the SRS where design decisions can be specified. That is, the quality requirement for abstraction is relaxed here. However, system constraints should only be included if they are truly required. —TPLT]

[This is where you would mention platform constrains, and licensing restrictions. —TPLT]

3 Stakeholder Requirements

[Technology-independent requirements. These come from statements made by the stakeholders. You will find these through interviews and surveys. They are often expressed in natural language. They are not necessarily testable, but they should be verifiable. —TPLT]

[The stakeholder requirements are numbered so that you can have traceability to other parts of your document. —TPLT]

SR1 TODO

SR2 TODO

SR3 TODO

[You should give your SRs meaningful labels, so that you can refer to them in other parts of the SRS. The label names should be meaningful mnemonics, not numbers. —TPLT]

4 User Personas

[Create 2-5 realistic personas. Personas are fictional characters that represent the different user types that might use your software. They are used to help understand the users' needs, experiences, behaviors and goals. They are not real people, but they should be based on real data collected from user research. —TPLT]

4.1 Persona 1

- Name:
- Description:
- Goals:
- Frustrations:
- Technical Ability:

4.2 Persona 2

- Name:
- Description:
- Goals:
- Frustrations:
- Technical Ability:

5 Use Cases

[Use cases are a way to describe the interactions between users and the system. They are typically written in natural language and describe the steps that a user takes to achieve a specific goal. Use cases can be used to identify functional requirements and to create test cases. —TPLT] [Typically 8-15 substantial use cases. —TPLT]

5.1 UC1: Example Use Case

Primary Actor

TODO

Preconditions

- TODO

Main Success Scenario

1. TODO
2. TODO
3. TODO

Alternative Flows

- TODO

Postconditions

- TODO

6 Functional Requirements

[Functional requirements describe what the system should do. They are typically expressed as a set of conditions or capabilities that the system must have. These requirements are usually derived from the stakeholder requirements and use cases. —TPLT]

[Use SHALL statements. Requirements must be atomic, testable, and traceable. —TPLT]

FR1: The system shall ...

FR2: The system shall ...

FR3: The system shall ...

[The requirements are given labels so that they can be referenced in other parts of the SRS. The label names should be meaningful mnemonics, not numbers. —TPLT]

7 Data Requirements

[The Data Requirements section describes what information the system must manage. It includes the major data entities, their attributes, and the relationships between them. It also includes a data dictionary that defines the meaning of each attribute. —TPLT]

7.1 Major Data Entities

[What information must exist for the system to accomplish its goals? —TPLT]

[For each entity, provide a short explanation. —TPLT]

- User
- Song
- Session
- Report

7.2 Data Dictionary

[For each entity, define all important attributes. —TPLT]

Attribute	Type	Description
user_id	UUID	Unique identifier
email	String	User email address

Table 2: Data Dictionary

7.3 Privacy and Data Retention

[Discuss storage, retention, deletion, privacy controls, consent, encryption, and compliance requirements. —TPLT]

TODO

8 Non-Functional Requirements

[You may have to add other NFRs. —TPLT]

[List your nonfunctional requirements. You may consider using a fit criterion to make them verifiable. —TPLT]

[The goal is for the nonfunctional requirements to be unambiguous, abstract and verifiable. This isn't easy to show succinctly, so a good strategy may be to give a "high level" view of the requirement, but allow for the details to be covered in the Verification and Validation document. —TPLT]

[An absolute requirement on a quality of the system is rarely needed. For instance, an accuracy of 0.0101 % is likely fine, even if the requirement is for 0.01 % accuracy. Therefore, the emphasis will often be more on describing now well the quality is achieved, through experimentation, and possibly theory, rather than meeting some bar that was defined a priori. —TPLT]

[You do not need an entry for correctness in your NFRs. The purpose of the SRS is to record the requirements that need to be satisfied for correctness. Any statement of correctness would just be redundant. Rather than discuss correctness, you can characterize how far away from the correct (true) solution you are allowed to be. This is discussed under accuracy. —TPLT]

NFR1: **Accuracy** [Characterize the accuracy by giving the context/use for the software. Maybe something like, "The accuracy of the computed solutions should meet the level needed for <engineering or scientific application>. The level of accuracy achieved by Software Engineering shall be described following the procedure given in Section X of the Verification and Validation Plan." If you do this provide a link to the VnV plan. —TPLT]

NFR2: **Usability** [Characterize the usability by giving the context/use for the software. You should likely reference the user characteristics section. The level of usability achieved by the software shall be described following the procedure given in Section X of the Verification and Validation Plan. Include a link to the VnV plan. —TPLT]

NFR3: **Maintainability** [The effort required to make any of the likely changes listed for Software Engineering should be less than FRACTION of the original development time. FRACTION is then a symbolic constant that can be defined at the end of the report. —TPLT]

NFR4: **Portability** [This NFR is easier to write than the others. The systems that Software Engineering should run on should be listed here. When possible the specific versions of the potential operating environments should be given. To make the NFR verifiable a statement could be made that the tests from a given section of the VnV plan can be successfully run on all of the possible operating environments. —TPLT]

- Other NFRs that might be discussed include verifiability, understandability, reusability, performance, reliability and availability, security, and accessibility. You may have to add other NFRs.

9 AI and Machine Learning Requirements

[Refer to Dr. Mosser's guest lecture on AI requirements. —TPLT]
[Include only when applicable. —TPLT]

9.1 Model Inputs

TODO

9.2 Model Outputs

TODO

9.3 Evaluation Metrics

[Accuracy, Precision, Recall, F1, BLEU, WER, etc. —TPLT]

TODO

9.4 Known Limitations

[Explicitly document expected failure modes. —TPLT]

TODO

10 Requirements Traceability Matrices

[The Requirements Traceability Matrix (RTM) is a table that shows the relationship between different types of requirements. It is used to ensure that all requirements are traceable and that no requirements are missed. The typical flow is: Stakeholder Need -¿ Use Case -¿ Requirement —TPLT]

[There is also a traceability matrix between functional and nonfunctional requirements. The purpose of some FRs is to satisfy NFRs. Therefore, it is important to show the relationship between the two. —TPLT]

[Further traceability to test cases and modules will be given in other documents. —TPLT]

Stakeholder Req	Use Case	Functional Req
SR1	UC1	FR1

Table 3: Traceability Between Stakeholder Requirements, Use Cases, and Functional Requirements

Functional Req	Nonfunctional Req
FR1	NFR1

Table 4: Traceability Between Functional and Nonfunctional Requirements

11 Rationale

[Provide a rationale for the decisions made in the documentation. Rationale should be provided for scope decisions, functional requirements, nonfunctioanl requirements, data requirements, and AI requirements. You don't need a rationale for everything, ust the major decisions that are not obvious. You should have traceability to the entities in the document that you are referencing. —TPLT]

12 Likely Changes

LC1: [Give the likely changes, with traceability to the entity that is likely to change. —TPLT]

13 Unlikely Changes

LC2: [Give the unlikely changes. The design can assume that the changes listed will not occur. —TPLT]

14 Appendices

14.1 Appendix A: Additional Materials

TODO

14.2 Appendix B: Development Plan

[This section is optional. It is used to explain the plan for developing the software. In particular, this section gives a list of the order in which the requirements will be implemented. In the context of a course this is where you can indicate which requirements will be implemented as part of the course, and which will be "faked" as future work. This section can be organized as a prioritized list of requirements, or it could show the requirements that will be implemented for "phase 1", "phase 2", etc. —TPLT]

14.3 Appendix C: Values of Auxiliary Constants

[Show the values of the symbolic parameters introduced in the report. —TPLT]

[The definition of the requirements will likely call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance. —TPLT]

[The value of FRACTION, for the Maintainability NFR would be given here. —TPLT]

TODO

14.4 Appendix D: Generative AI Disclosure

[Provide full transparency on how generative AI was used in the creation of this document. This includes any text, figures, or tables that were generated by AI. If any skills were created for AI agents to review the documentation, those should be disclosed here. —TPLT]

14.5 Appendix E: Reflection

[This isn't part of the SRS, but it is part of the course. —TPLT]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?